

FlowForge: A Visual ETL Pipeline Builder

Applied Cloud Programming (ACP) — CW3

University of Edinburgh | UID: S2816523

1 The Problem

Building a data pipeline today requires writing Kafka configuration, Flink streaming jobs, serialisation formats, and managing service deployments—all before a single byte of data moves. Non-technical users are locked out entirely; even engineers spend most of their time on infrastructure *plumbing* rather than on insight.

Enterprise tools such as AWS Glue [1] or Talend solve this with visual interfaces, but cost thousands a month. Apache NiFi [2] is self-hosted but targets enterprise data routing, not a Kafka/Flink/S3 stack usable by non-engineers. No lightweight, self-hosted alternative covering this combination exists, likely making this space dominated by expensive enterprise platforms.

2 The Idea

FlowForge is a drag-and-drop ETL (Extract–Transform–Load) pipeline builder. Users drag *source*, *transform*, and *sink* nodes onto a canvas, connect them with edges, configure each node by clicking on it, and press **Run**. The system translates that visual graph into a real, executing cloud data pipeline—no code required.

Who benefits:

- **Data analysts** who understand their data but cannot write Flink or Kafka code.
- **Small engineering teams** needing one-off data migrations without enterprise infrastructure.
- **Students and practitioners** learning cloud data concepts through hands-on, visual experimentation.

Concretely: moving weekly S3 transaction records into Postgres typically requires a developer to write and maintain a Flink job; FlowForge reduces this to minutes. An **AI assistant** further lets users describe the pipeline in plain English and receive a ready-to-run canvas automatically.

3 Architecture & Design

Two-Plane Architecture

The central design decision separates orchestration from data movement:

- **Control plane (RabbitMQ [3]):** Tiny coordination messages (start, complete, fail-and-retry)¹ that must be *reliable* and *acknowledged*.
- **Data plane (Kafka [4]):** The actual records streaming between nodes. Each pipeline edge gets its own dedicated Kafka topic, so independent branches run in parallel.

Execution

When the user clicks **Run**, the backend: (1) validates the graph is a cycle-free DAG² via depth-first search; (2) creates a Kafka topic per edge; (3) writes an audit entry to DynamoDB [5]; (4) dispatches root nodes to RabbitMQ; (5) triggers each downstream node once all its dependencies complete;

¹Messages exceeding the retry limit are routed to a Dead-Letter Queue (DLQ) rather than silently dropped, enabling inspection and re-driving of failed tasks.

²Directed Acyclic Graph—depth-first search detects any cycle before execution begins.

and (6) pushes live status to the browser via WebSocket—users watch nodes light up in real time (see Figures 1 and 2).

Eight node types are fully implemented (Figure 3): two sources (S3, Kafka), three transforms (Filter, Field Mapper, Aggregator), and three sinks (PostgreSQL, S3, Kafka). Filter and Aggregator submit a bounded Flink [6] streaming job via the REST API, falling back to in-process execution if Flink is unreachable—a deliberate resilience decision. AI generation uses a three-level fallback: Ollama [7] LLM → keyword parser → static demo pipeline.

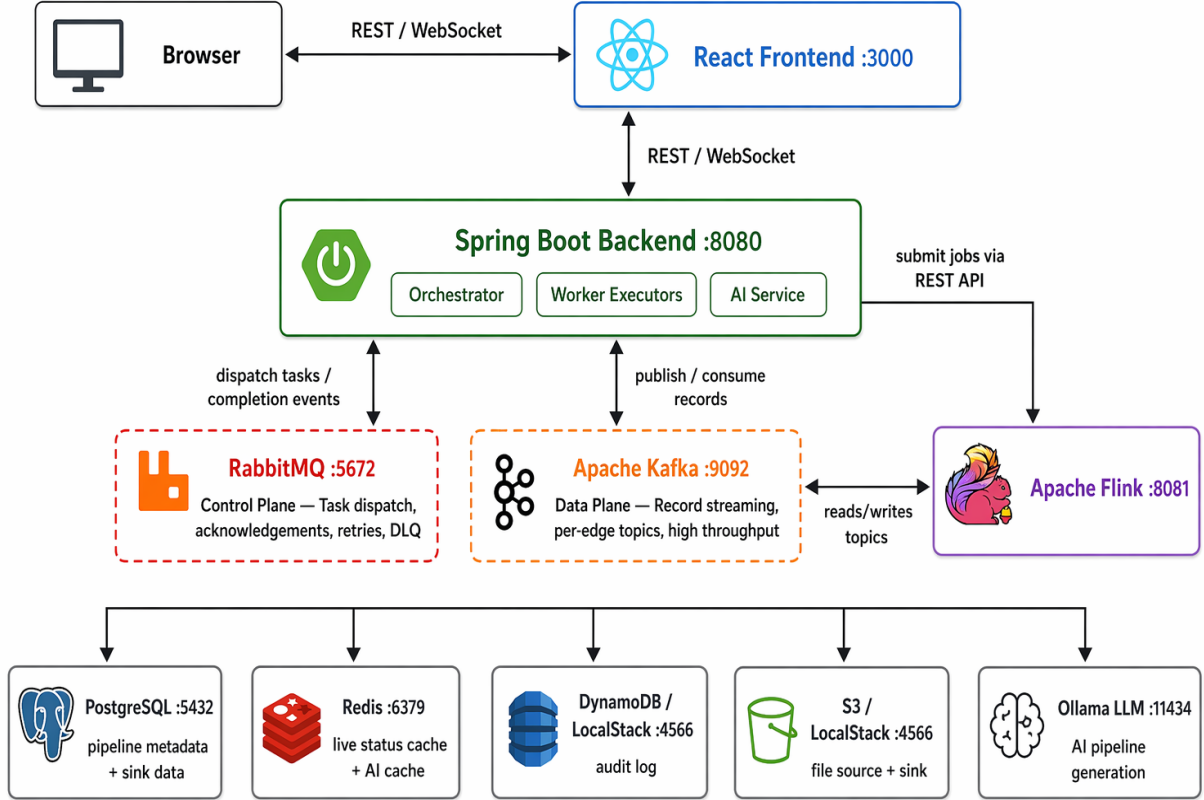


Figure 1: FlowForge system architecture: RabbitMQ routes orchestration commands; Kafka carries data records between nodes.

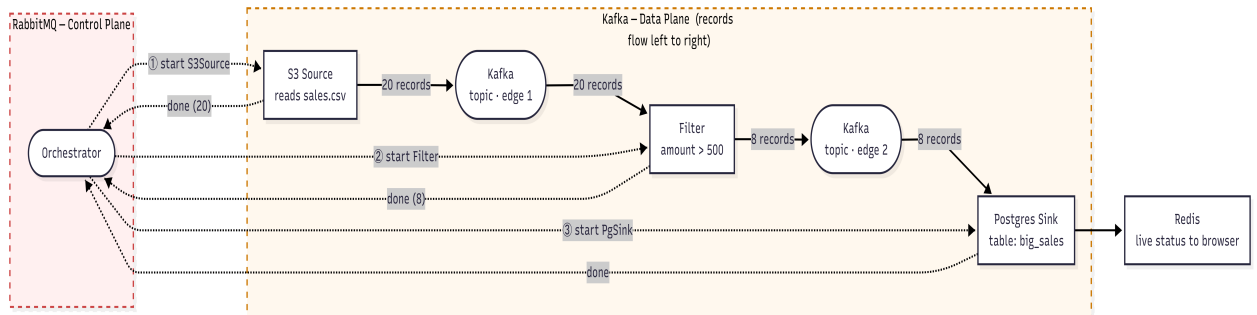


Figure 2: Concrete execution of an S3 Source → Filter → Postgres Sink pipeline.

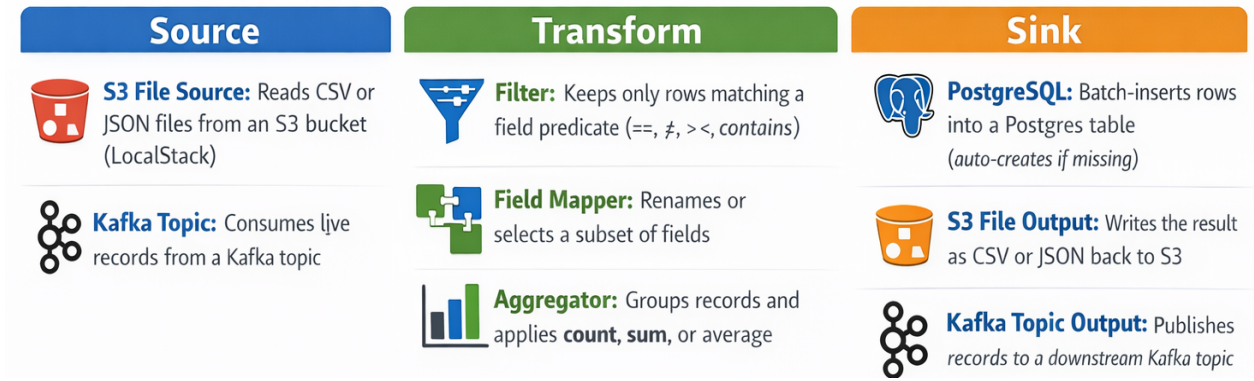


Figure 3: All eight implemented node types supporting ETL.

4 Technologies & Engineering Process

Technology	Reason for choice
RabbitMQ [3]	Task routing with acknowledgements, retries, and a DLQ for failed nodes.
Kafka [4]	High-throughput ordered streaming; per-edge topics for parallel branch execution.
Apache Flink [6]	Bounded-source engine—stops automatically once all input records are consumed.
Redis [8]	Sub-millisecond live status reads; caches AI pipelines by SHA-256 prompt hash.
DynamoDB [5]	Append-only audit log, kept separate from the operational Postgres database.
LocalStack [9]	Real S3 and DynamoDB APIs locally—no cloud account or cost required.
Ollama [7]	Local LLM inference (Phi-3 Mini). No API cost; falls back gracefully if unavailable.

Design Choices

The two-plane split was a deliberate architectural decision: RabbitMQ’s acknowledgement model gives reliable, exactly-once task delivery for orchestration, while Kafka’s high-throughput log handles the actual data records. Mixing both concerns into a single broker would couple retry logic to record streaming. Per-edge Kafka topics give natural data isolation and let independent pipeline branches execute in parallel without coordination. Redis was chosen for live status over Postgres because sub-millisecond reads keep WebSocket updates immediate; DynamoDB provides an immutable audit log without polluting the operational database.

Engineering Process

SDLC. Development followed a four-phase iterative cycle within the 12–16 hour budget: (1) AI-generated architectural scaffold and planning document; (2) service-by-service integration, wiring each component into Docker Compose; (3) diagnose and fix failing integrations; (4) end-to-end pipeline verification against seeded data.

What AI produced. Claude generated the architectural plan, all seven Spring Boot configuration classes, Docker Compose wiring, and the eight React node components. The scaffold compiled and the UI rendered on the first attempt, covering roughly the first two hours of development.

Where AI went wrong.

- **Flink REST endpoint:** Generated code polled `/jobs/{id}/overview`—invalid in Flink 1.18. Fixed to `/jobs/{id}` after reading Flink logs.
- **Kafka consumer timing:** Polled before partition assignment completed (3–5 s rebalance delay), reading zero records. Fixed with a 30-second wait loop and `seekToBeginning` [4].
- **LocalStack SDK wiring:** AWS clients used the default endpoint. Fixed by overriding `endpointOverride` per LocalStack documentation [9].

QA. No automated test suite—a deliberate scope trade-off for a PoC. Each node type was verified end-to-end against seeded `sales.csv` data, status badges were confirmed to progress *WAITING* → *RUNNING* → *COMPLETED*, and the DLQ path was tested by targeting a non-existent S3 key.

5 Conclusion

FlowForge demonstrates that a self-hosted, visually accessible ETL tool can be built on production-grade cloud technologies. The two-plane architecture correctly separates orchestration from data movement. All eight node types work end-to-end; Filter and Aggregator use Flink with graceful fallback. The AI generation feature makes the system genuinely more useful than hand-writing pipeline code.

Known trade-offs: orchestrator state is in-memory and lost on restart; pipelines use a synchronous Kafka drain rather than true streaming; there is no authentication layer. These are deliberate PoC decisions, not oversights.

References

- [1] Amazon Web Services. *AWS Glue – Serverless Data Integration Service*. 2024. URL: <https://aws.amazon.com/glue/> (visited on 04/13/2026).
- [2] Apache Software Foundation. *Apache NiFi – An Easy to Use, Powerful, and Reliable System to Process and Distribute Data*. 2024. URL: <https://nifi.apache.org/> (visited on 04/13/2026).
- [3] Broadcom Inc. *RabbitMQ – One Broker to Queue Them All*. 2024. URL: <https://www.rabbitmq.com/> (visited on 04/13/2026).
- [4] Apache Software Foundation. *Apache Kafka – A Distributed Streaming Platform*. 2024. URL: <https://kafka.apache.org/documentation/> (visited on 04/13/2026).
- [5] Amazon Web Services. *Amazon DynamoDB – Fast, Flexible NoSQL Database Service*. 2024. URL: <https://aws.amazon.com/dynamodb/> (visited on 04/13/2026).
- [6] Apache Software Foundation. *Apache Flink – Stateful Computations over Data Streams*. 2024. URL: <https://flink.apache.org/> (visited on 04/13/2026).
- [7] Ollama. *Ollama – Run Large Language Models Locally*. 2024. URL: <https://ollama.com/> (visited on 04/13/2026).
- [8] Redis Ltd. *Redis – The Real-time Data Platform*. 2024. URL: <https://redis.io/> (visited on 04/13/2026).
- [9] LocalStack GmbH. *LocalStack – A Fully Functional Local Cloud Stack*. 2024. URL: <https://localstack.cloud/> (visited on 04/13/2026).

A AI Usage

AI assistance (Claude) was used in three ways: (1) generating an architectural planning document describing the system designs *before* any code was written; (2) producing boilerplate for configuration classes, Docker Compose wiring, and frontend node components; and (3) helping debug integration issues between Spring Boot and the AWS SDK for LocalStack. All design decisions—the two-plane architecture, the Flink-with-fallback pattern, the per-edge Kafka topic strategy, the three-level AI generation fallback (LLM → keyword parser → static demo)—were made and implemented by me. AI accelerated delivery within the 12–16 hour scope; it did not replace understanding.

B Build & Run Instructions

Prerequisites

- Docker Desktop (or Docker Engine + Compose v2), at least version 24
- 8 GB RAM recommended (Flink + Kafka are memory-hungry)

- Ports free: 3000, 8080, 5432, 5672, 6379, 9092, 8081, 4566, 11434, 15672

Step 1 — Start the full stack

```
cd flowforge
docker compose up --build -d
```

Wait **90 seconds** for all services to initialise. On the first run, Docker builds the Flink JAR and Ollama downloads `phi3:mini`—this adds a few minutes. Subsequent starts are fast.

```
docker compose ps    # all services should show "healthy" or "running"
```

Step 2 — Seed test data

```
bash scripts/init-localstack.sh    # creates S3 buckets + DynamoDB table
bash scripts/seed-data.sh          # uploads sales.csv and products.json
```

Step 3 — Open the application

Navigate to <http://localhost:3000>. A demo pipeline (S3 Source → Filter → Postgres Sink) is pre-loaded on the canvas. Click any node to configure it, press **Save** then **Run Pipeline**. Status badges update live.

Service URLs

Service	URL	Credentials
Frontend	http://localhost:3000	—
Backend API	http://localhost:8080/api	—
RabbitMQ Management	http://localhost:15672	flowforge / flowforge
Flink Dashboard	http://localhost:8081	—
LocalStack (S3)	http://localhost:4566	test / test

Stopping

```
docker compose down    # stops and removes containers
docker compose down -v # also removes all stored data (full reset)
```